

v1.0 · 2025

Mall Delivery App

Developer Handbook

A step-by-step guide for interns building the functional prototype



Customer



Store



Runner



Rider

React Native + Node.js + PostgreSQL + Socket.io

Contents

01 Introduction & Overview

02 Recommended Tech Stack

03 Architecture

- System diagram
- Request lifecycle
- Folder structure

04 Local Setup

- Prerequisites
- Step-by-step guide
- Environment variables

05 Flow 1 — Customer

- Journey steps
- Screens to build
- Try & Buy timer

06 Flow 2 — Store (Retailer)

- Journey steps
- Screens & realtime orders

07 Flow 3 — Mall Runner

- Journey steps
- Status transitions

08 Flow 4 — Delivery Rider

- Journey steps
- Live location sharing

09 Data Models

- Prisma schema
- Relationships

10 API Reference

- Auth
- Products
- Orders
- Store
- Runner / Rider

11 Realtime & Sockets

- Socket events
- Room strategy

12 State Management

- Zustand
- React Query

13 Sprint 1 Tasks

- Backend tasks
- Frontend tasks

- Definition of done

14 Code Conventions

- Naming
- Git workflow
- Error handling

01

Introduction & Overview

Mall Delivery App

Developer Handbook · For interns building the functional prototype

This handbook is your single source of truth for building the mall delivery prototype. It covers all four user journeys — Customer, Store, Mall Runner, and Delivery Rider — and walks you through every layer of the stack, from database models to mobile screens.

You don't need to know everything before you start. Use this as a reference — come back to a section when you're working on that part of the app.

What we're building

PRODUCT	Instant mall delivery app. Customers browse, order, and receive items from mall stores — with a 10-minute try-before-you-buy window at the door.
PROTOTYPE GOAL	Functional end-to-end flow. All 4 actors (Customer, Store, Runner, Rider) can interact live. Real state changes. No hard-coded mocks for core journeys.

The four actors

Actor	Role	Platform	Primary actions
■■ Customer	Browses and orders	Mobile app	Search, cart, checkout, track, try & buy
■ Store	Prepares items	Web / tablet	Receive orders, hand to runner, manage inventory
■ Mall runner	Collects from store	Mobile app	Accept, collect item, handover to rider
■ Delivery rider	Last-mile delivery	Mobile app	Accept, pick up from runner, deliver, handle return

Tip Start with the Customer flow. It touches every other actor and will force you to think about the full data model early.

02

Recommended Tech Stack

Since the stack isn't locked yet, here's the recommended setup for a beginner-friendly, production-ready prototype. Each choice is explained so you understand the why.

Frontend

MOBILE	React Native (Expo) for iOS + Android + web from one codebase. Expo Go lets you test on a real device without Xcode or Android Studio. React Navigation for routing. Tailwind (NativeWind) for styling.
STORE DASHBOARD	React (web) for the store tablet interface. Vite for fast builds. Tailwind CSS for styling. Connects to the same backend API.

Backend

SERVER	Node.js + Express — approachable for beginners, vast ecosystem. Prisma ORM for safe, readable database queries. Socket.io for real-time order events. JWT + bcrypt for authentication.
--------	--

Infrastructure

HOSTING	Supabase for hosted PostgreSQL + built-in auth (free tier). Railway for deploying the Node server (free tier). Cloudinary for product images. GitHub + Vercel for the web dashboard.
---------	--

Alternatives if you already have a preference

Layer	Recommended	Acceptable alternative
Mobile	React Native / Expo	Flutter (Dart)
Backend	Node.js / Express	Python / FastAPI
Database	PostgreSQL	MySQL
ORM	Prisma	TypeORM, Sequelize
Realtime	Socket.io	Supabase Realtime
Auth	JWT + bcrypt	Supabase Auth, Clerk

Note Why not Firebase? Firebase is a NoSQL document store — relational data like orders, riders, and stores is tricky to model correctly. PostgreSQL with Prisma is only slightly more work and teaches transferable skills.

03

Architecture

The app has three layers: a mobile/web frontend, a REST + WebSocket backend, and a database. Understanding how these connect is the most important thing before writing any code.

Request lifecycle — a typical order

1 Customer taps "Add to cart"

React Native calls POST /api/orders with product ID + customer token. Token is attached in the HTTP Authorization header.

2 API server validates the request

JWT middleware decodes the token, finds the customer in the DB. Express route handler runs, creates the order in PostgreSQL via Prisma.

3 Server emits a Socket.io event

socket.to(storeRoom).emit("new_order", orderData) — the store dashboard receives it instantly without polling.

4 Order status updates cascade

Every status change (picked, collected, out_for_delivery) emits a new socket event. The customer's live tracking screen listens for order_update events.

5 Try & Buy timer starts at delivery

When rider marks ARRIVED, server sets tryTimerEnd = now + 10 min and emits try_timer_start. Frontend shows countdown. After 10 min, KEEP or RETURN.

Folder structure

```
mall-delivery/ ■■■ backend/ ■ ■■■ prisma/ ■ ■ ■■■ schema.prisma # DB schema – source
of truth ■ ■■■ src/ ■ ■ ■■■ routes/ # auth, orders, store, runner, rider ■ ■ ■■■
middleware/ # JWT auth, error handler ■ ■ ■■■ sockets/ # socket.io event handlers ■ ■
■■■■ index.js # entry point ■ ■■■ package.json ■■■ mobile/ # React Native (Expo) ■ ■■■
src/ ■ ■ ■■■ screens/ # one folder per actor ■ ■ ■■■ navigation/ ■ ■ ■■■ store/ #
Zustand global state ■ ■ ■■■ api/ # axios instance + query hooks ■ ■ ■■■ components/ ■
■■■■ app.json ■■■ web/ # Store dashboard (React) ■■■ src/
```

04

Local Setup

Follow these steps in order. Don't skip any. If something fails, check the step carefully before asking for help — the error message usually tells you exactly what's wrong.

Prerequisites

Tool	Version	Install
Node.js	≥ 18	nodejs.org/en/download
npm	≥ 9	Comes with Node
Git	any	git-scm.com
Expo Go	latest	App Store / Play Store (on your phone)
VS Code	any	code.visualstudio.com

Step-by-step

1 Clone the repo

```
git clone https://github.com/your-org/mall-delivery.git cd mall-delivery
```

2 Set up the backend

```
cd backend && npm install cp .env.example .env # fill in Supabase URL + password npx prisma generate npx prisma migrate dev # creates all tables npm run dev # starts on port 3001
```

3 Set up the mobile app

```
cd ../mobile && npm install cp .env.example .env # set API_URL to your local IP e.g. http://192.168.x.x:3001 npx expo start # scan QR code with Expo Go
```

4 Seed the database

```
cd backend && npx prisma db seed # Creates 3 test stores, 20 products, one account per role. # Credentials are in backend/prisma/seed.js
```

Tip Use your machine's local IP address (not localhost) for API_URL — your phone and laptop must be on the same WiFi network.

Environment variables

```
# backend/.env DATABASE_URL="postgresql://user:pass@host/dbname"
JWT_SECRET="your-secret-key-min-32-chars" PORT=3001 CLOUDINARY_URL="cloudinary://..." #
mobile/.env EXPO_PUBLIC_API_URL="http://192.168.1.x:3001"
```

Note Never commit your .env file. It's already in .gitignore — don't remove it.

05

Flow 1 — Customer

The most complex flow. The customer is the "source of truth" for an order — everything else is triggered by their actions.

Journey

Step	Action	Decision / branch
1	Push notification → App opens	—
2	Home feed / Trending	—
3	Search or browse by store / category	—
4	Product detail page	Size unavailable → Notify Me
5	Select size + Add to cart	—
6	Delivery time estimate	High ETA → Notify Me
7	Checkout + Payment	—
8	Order confirmed — XX min ETA	—
9	Live tracking	—
10	Rider arrives at door	—
11	Try & Buy — 10 min timer	Keep → payment finalised Return → refund / new order

Screens to build

Screen	Key components	Data source
HomeScreen	Trending grid, category tabs	GET /products?trending=true
SearchScreen	Search bar, filters, results list	GET /products?q=&store;=
ProductDetailScreen	Images, size selector, ETA, CTA	GET /products/:id
CartScreen	Item list, total, delivery estimate	Local state
CheckoutScreen	Address confirm, payment (mock)	POST /orders
OrderConfirmScreen	ETA, order ID, track CTA	Order from response
LiveTrackingScreen	Map, status timeline, rider info	WS: order_update
TryBuyScreen	10-min countdown, Keep/Return	WS: try_timer_start

Try & Buy timer — frontend code

```
// TryBuyScreen.js
const [seconds, setSeconds] = useState(600);
useEffect(() => {
  const timer = setInterval(() => {
    setSeconds(s => {
      if (s <= 1) {
        clearInterval(timer);
        handleAutoReturn();
        return 0;
      }
      return s - 1;
    });
  }, 1000);
  return () => clearInterval(timer);
}, []);
// Display as MM:SS
const display = `${Math.floor(seconds/60)}:${String(seconds%60).padStart(2, "0")}`;
```

Tip For the prototype, mock push notifications with an in-app alert. Implement real push (Expo Notifications) only after the core flow works.

06

Flow 2 — Store (Retailer)

The store is a web dashboard (tablet-friendly). Their job: receive an order notification, prepare the item, hand it to the runner, and mark it as picked.

Step	Action
1	Store onboarding — grant POS/API access or manual upload
2	Inventory synced (real-time or manual)
3	Customer orders item → new order notification arrives on dashboard
4	Mall runner comes to store
5	Store staff hands over item + bill, marks order as Picked in system
6	Order moves to runner → delivery agent
7	Monthly settlement & commission payment

Screens to build

Screen	Key components	Data
OrdersBoard	Real-time incoming orders, status columns	WS: new_order
InventoryManager	Product list, stock count, upload CSV	GET/PUT /store/inventory
OrderDetail	Item photo, size, runner ETA, mark-picked CTA	GET /orders/:id
SettlementView	Monthly earnings, commission breakdown	GET /store/settlement

Real-time order board — socket code

```
// Store dashboard — listen for new orders
useEffect(() => {
  socket.emit("join_store_room", storeId);
  socket.on("new_order", (order) => {
    setOrders(prev => [order, ...prev]);
    playAlertSound();
  });
  socket.on("runner_arrived", ({
    orderId
  }) => {
    updateOrderStatus(orderId, "RUNNER_HERE");
  });
  return () => {
    socket.off("new_order");
  }, [storeId]);
}, [storeId]);
```

07

Flow 3 — Mall Runner

The runner is physically inside the mall. They pick up items from stores and hand them to the delivery rider at the mall gate. Accept, collect, handover — that's their whole loop.

Step	Action
1	Runner logs in (inside mall)
2	New order notification received
3	Accept order — view order details + store location
4	Go to store, collect item from staff
5	Mark as Collected in app
6	Move to mall gate / staging point
7	Handover to delivery rider, order status updated
8	Ready for next order

Status transitions

```
// Runner moves an order through these statuses: PENDING → RUNNER_ASSIGNED // runner
accepts the order RUNNER_ASSIGNED → COLLECTED // runner marks item collected from store
COLLECTED → HANDED_TO_RIDER // runner confirms handover at mall gate
```

Screens to build

Screen	Key components	Data
RunnerDashboard	Available orders, current job, earnings	GET /runner/orders
AcceptOrderScreen	Store name, item, distance, accept/decline	WS: runner_order_available
CollectionScreen	Store map pin, item checklist, mark collected	PUT /orders/:id/status
HandoverScreen	Rider details, meeting point, confirm	PUT /orders/:id/status

Tip The runner's app can be very minimal for the prototype — a list + action buttons is fine. Focus on getting the status transitions correct first.

08

Flow 4 — Delivery Rider

The rider does last-mile delivery. They receive the item from the runner at the mall gate, deliver to the customer's door, and manage the Try & Buy outcome.

Step	Action	Branch
1	Rider accepts order via app	—
2	Go to mall gate / staging point	—
3	Receive item from mall runner	—
4	Start delivery to customer (live location sharing)	—
5	Reach customer location	—
6	Customer Try & Buy — 10 min timer starts	—
7	Customer decides	Keep → Mark Delivered Return → Take item back
8 (return)	Return to mall / hub, mark Return in app	—

Live location sharing

```
// Rider app: emit GPS every 5 seconds during active delivery
import * as Location from "expo-location";
useEffect(() => {
  let interval;
  if (isDeliveryActive) {
    interval = setInterval(async () => {
      const { coords } = await Location.getCurrentPositionAsync({});
      socket.emit("rider_location", {
        orderId,
        lat: coords.latitude,
        lng: coords.longitude,
      });
    }, 5000);
  }
  return () => clearInterval(interval);
}, [isDeliveryActive]);
```

Full status machine (all actors)

```
PENDING → RUNNER_ASSIGNED // runner accepts
RUNNER_ASSIGNED → COLLECTED // runner collects from store
COLLECTED → HANDED_TO_RIDER // runner hands to rider at gate
HANDED_TO_RIDER → OUT_FOR_DELIVERY // rider confirms pickup
OUT_FOR_DELIVERY → ARRIVED // rider taps "I'm at the door"
ARRIVED → DELIVERED // customer keeps
ARRIVED → RETURNING // customer returns
RETURNING → RETURNED // rider back at hub, item logged
ARRIVED / RETURNED → REFUNDED // refund initiated
```

09

Data Models

This is your Prisma schema — the source of truth for your database. Every table, every relationship. Run `npx prisma migrate dev` after any change.

User model

```
model User { id String @id @default(uuid()) email String @unique phone String role Role
// CUSTOMER | RUNNER | RIDER | STORE_STAFF password String // bcrypt hash orders Order[]
createdAt DateTime @default(now()) }
```

Product & Order models

```
model Product { id String @id @default(uuid()) name String price Decimal images String[]
storeId String store Store @relation(fields: [storeId], references: [id]) variants
Variant[] // sizes, colours category String trending Boolean @default(false) } model
Order { id String @id @default(uuid()) customerId String customer User @relation(fields:
[customerId], references: [id]) items OrderItem[] status OrderStatus runnerId String? //
assigned after PENDING riderId String? // assigned after COLLECTED deliveryAddr String
etaMinutes Int tryTimerEnd DateTime? // set when rider marks ARRIVED createdAt DateTime
@default(now()) } enum OrderStatus { PENDING RUNNER_ASSIGNED COLLECTED HANDED_TO_RIDER
OUT_FOR_DELIVERY ARRIVED DELIVERED RETURNING RETURNED REFUNDED }
```

Relationships

Model	Belongs to	Has many
Product	Store	Variants, OrderItems
Order	Customer (User)	OrderItems
Order	Runner (User) — optional	—
Order	Rider (User) — optional	—
Store	—	Products, StoreStaff
Variant	Product	—
OrderItem	Order, Product, Variant	—

10

API Reference

All endpoints are prefixed with /api. Protected routes require an Authorization: Bearer header. Responses follow the shape { data, error }.

Auth

POST	/auth/register	Create account. Body: email, password, phone, role
POST	/auth/login	Returns JWT token. Body: email, password
GET	/auth/me	Returns current user from token ■

Products

GET	/products	Query params: q, storeId, category, trending
GET	/products/:id	Product detail with variants and store info
POST	/products/:id/notify	Subscribe to restock alert ■

Orders

POST	/orders	Create order. Body: items[], deliveryAddress ■
GET	/orders/:id	Order detail + current status ■
PUT	/orders/:id/status	Update status. Body: status, actorId ■
POST	/orders/:id/outcome	KEEP or RETURN after Try & Buy ■

Store

GET	/store/orders	Active orders for this store ■
GET	/store/inventory	All products and stock levels ■
PUT	/store/inventory/:id	Update stock count ■
GET	/store/settlement	Monthly earnings summary ■

Runner & Rider

GET	/runner/orders	Available + active orders for runner ■
GET	/rider/orders	Available + active orders for rider ■

■ = requires Authorization: Bearer header

11

Realtime & Sockets

Socket.io powers all live updates — order notifications, location tracking, and the Try & Buy timer. Actors join 'rooms' based on their role and ID so events reach the right device.

Socket events reference

Event name	Direction	Payload	Who listens
new_order	server → client	full order object	Store dashboard
runner_order_available	server → client	order summary	Runner app
rider_order_available	server → client	order summary	Rider app
order_update	server → client	{orderId, status}	Customer app
rider_location	client → server	{orderId, lat, lng}	Server → customer
rider_location_update	server → client	{lat, lng}	Customer tracking screen
try_timer_start	server → client	{orderId, endsAt}	Customer + rider
runner_arrived	server → client	{orderId}	Store dashboard

Room strategy

```
// When a user authenticates via socket: socket.join(`user:${userId}`); // personal room
socket.join(`store:${storeId}`); // store staff room
socket.join(`order:${activeOrderId}`); // order-specific updates // Server emits to the
right room: io.to(`store:${storeId}`).emit("new_order", order);
io.to(`order:${orderId}`).emit("order_update", { status });
io.to(`user:${customerId}`).emit("rider_location_update", { lat, lng });
```

Tip For the prototype, a simple room-based approach is enough. Don't over-engineer. Optimise later if you hit performance issues.

12

State Management

Use Zustand for global client state — simpler than Redux and perfect for a prototype. Use React Query (TanStack Query) for server data — it handles caching and refetching automatically.

What goes where

Data	Tool	Why
Cart items, active order, auth user	Zustand	Needs to persist across screens, no server sync
Products, order history, store data	React Query	Needs caching, loading/error states, refetch
Form inputs, toggles	useState	Local to one component
Socket connection	Context + useRef	One connection shared app-wide

Zustand store example

```
// store/useOrderStore.js import { create } from "zustand"; export const useOrderStore = create((set) => ({ activeOrder: null, cart: [], addToCart: (item) => set((state) => ({ cart: [...state.cart, item] })), setActiveOrder: (order) => set({ activeOrder: order }), clearCart: () => set({ cart: [] }), }));
```

React Query for API calls

```
// Fetches product, caches it, auto-refetches if stale import { useQuery } from "@tanstack/react-query"; import { api } from "../api/client"; export function useProduct(productId) { return useQuery({ queryKey: ["product", productId], queryFn: () => api.get(`/products/${productId}`).then(r => r.data), }); }
```

13

Sprint 1 Tasks

These are the tasks for the first sprint — roughly 2 weeks. Focus on getting data flowing end-to-end before worrying about polish.

Backend tasks

1 Set up Express + Prisma

Init project, connect to Supabase, run first migration with User + Product + Order models.

2 Auth endpoints

Build POST /auth/register and /auth/login with bcrypt + JWT. Test with Postman before moving on.

3 Order lifecycle endpoints

POST /orders and PUT /orders/:id/status — the backbone of the whole app.

4 Socket.io setup

Basic room joining on auth. Emit new_order when an order is created. Test with a simple HTML test client.

5 Seed data

Write a seed script with 2-3 stores, 10+ products, and one test account per role.

Frontend tasks

6 Expo project + navigation shell

Set up React Native + Expo. Build a tab navigator with placeholder screens for the Customer flow.

7 Auth screens + token storage

Login / register forms. Store JWT in expo-secure-store. Redirect to home on login.

8 Home + product screens

Fetch and display products. Build product detail with size selector. Hardcode delivery ETA for now.

9 Checkout → order confirmed flow

Place a real order via the API. Show confirmation screen with order ID.

10 Store dashboard (web)

Simple React page that connects to socket and shows incoming orders in real time. Mark-as-picked button.

Sprint 1 definition of done

A customer can place an order on the mobile app → the store dashboard shows the order in real-time → the customer sees the status update on their screen. Full mock data is fine for runner/rider in Sprint 1.

14

Code Conventions

Everyone writes code that looks like it came from the same person. These rules are not optional — they keep the codebase clean when multiple interns work simultaneously.

Naming conventions

Thing	Convention	Example
Components	PascalCase	ProductDetailScreen
Functions & variables	camelCase	handleAddToCart
Files	camelCase	useOrderStore.js
API routes	kebab-case, plural nouns	/api/orders/:id
Database columns	camelCase (Prisma)	createdAt, storeId
Socket events	snake_case	new_order, try_timer_start
Environment variables	SCREAMING_SNAKE_CASE	JWT_SECRET, DATABASE_URL

Git workflow

1 One branch per task

Branch format: feature/task-name e.g. feature/order-status-api

2 Small, frequent commits

Commit after each working change. Format: feat: add order status endpoint or fix: correct JWT expiry

3 Pull request to main

Never push directly to main. Open a PR and ask a teammate to review. At least one approval before merge.

Error handling patterns

```
// Wrap async route handlers to avoid unhandled rejections const asyncHandler = (fn) =>
(req, res, next) => Promise.resolve(fn(req, res, next)).catch(next); // Return consistent
error shapes across all endpoints res.status(400).json({ error: "Size not available",
code: "SIZE_UNAVAILABLE" }); // On the frontend, always handle loading + error states
const { data, isLoading, error } = useProduct(id); if (isLoading) return <LoadingSpinner
/>; if (error) return <ErrorMessage message={error.message} />;
```

Tip If you're unsure about something, ask before writing it. A 5-minute question saves hours of refactoring later.